

BRSR Sustainability Analyst Agent

A tool-calling AI agent that answers natural-language questions over Indian SEBI Business Responsibility and Sustainability Report (BRSR) filings, with grounded SQL retrieval, PDF citations, and cloud deployment.

5,795

REPORTS INDEXED

1,696

COMPANIES

2020–26

FILING YEARS

4

AGENT TOOLS

Package: sustally-brsr-agent · v1.0.0

Repository folder: stage3

Stack: Node.js · Express · SQLite / Neon Postgres · Ollama / OpenAI / OpenRouter

Hosting: Vercel (serverless) · Firebase Auth · Hugging Face / Cloudflare R2 PDFs

Document date: 17 July 2026

Contents

1. Project Overview & Explanation

2. System Architecture

3. Modules & Codebase Structure

4. How a Query Works

5. Project Setup

6. Hosting & Deployment

7. Tools Used and Why

8. What I Learned from This Project

9. Evaluation, Testing & Quality

10. Appendix — Schema, Scripts & API

1. Project Overview & Explanation

SusTally BRSR Agent is a full-stack sustainability analyst that turns SEBI BRSR/XBRL filings into a queryable knowledge base and lets users ask ESG questions in plain English.

Indian listed companies file **Business Responsibility and Sustainability Reports (BRSR)** with the National Stock Exchange (NSE). These filings contain greenhouse-gas emissions, energy and water use, waste, workforce diversity, board composition, and safety metrics. The raw format is XBRL/XML — useful for machines, difficult for analysts to explore interactively.

This project solves that gap. It downloads filings, parses XBRL into a structured `reports` table, and exposes a chat interface where a large language model (LLM) decides which database tools to call. Answers are grounded in real numbers, optionally cite PDF page sources, and can include charts for rankings and comparisons.

1.1 Problem statement

- BRSR data is scattered across thousands of XBRL and PDF files.
- Analysts need cross-company rankings, year filters, and sector comparisons — not one PDF at a time.
- LLMs alone hallucinate ESG numbers; they need controlled retrieval and schema-bound rules.
- Production use requires auth, durable chat history, and public PDF links for citations.

1.2 Solution in one sentence

Design principle. Prefer one safe SQL tool call when possible; never invent metrics or columns; cite PDF pages only when a verified page index exists; stream progress so the UI feels responsive.

1.3 Current corpus (local SQLite snapshot)

METRIC	VALUE
Indexed reports	5,795
Unique companies	1,696
Year range	2020 – 2026
Peak filing years	2023 (1,385), 2024 (1,420), 2025 (1,320)
Default local model	<code>qwen2.5:7b-instruct</code> via Ollama
Cloud models	OpenAI (e.g. gpt-4o-mini) or OpenRouter

2. System Architecture

The system is a classic three-tier design: a browser UI, an Express API that hosts the agent loop, and a dual-mode database (SQLite locally, Neon Postgres in production). LLM inference is pluggable — Ollama locally, OpenAI or OpenRouter in the cloud.



Figure 1. High-level architecture of the SusTally BRSR Agent.

2.1 Architectural style – tool-calling agent

Unlike a fixed classify → SQL → narrate pipeline, this agent gives the LLM four tools and lets it decide when and how to query. A compact system prompt keeps context small for faster inference. Guardrails in code (SELECT-only SQL, unknown-column checks, ranking sanitization, citation verification) compensate for model mistakes.

2.2 Data flow (ingest → answer)

```
# Offline / batch pipeline
NSE filings → download_nse_reports.js → data/xbrl + data/pdf
XBRL XML → preprocess.js → reports table (SQLite/Postgres)
PDF files → upload_pdfs_to_hf/r2 → public URLs + pdf_*_urls.json
PDF text → page-index.js → metric_pages_json (citations)

# Online / request path
User question → POST /api/chat (SSE)
                → runAgent() tool loop
                → enrich + optional PDF verify
                → streamed markdown answer (+ json-chart)
```

3. Modules & Codebase Structure

```
stage3/
├── package.json / vercel.json / .env.example
├── agent/
│   ├── src/                                # runtime
│   │   ├── server.js                      # Express API, SSE, auth, uploads
│   │   ├── agent.js                      # tool-calling loop
│   │   ├── system-prompt.js             # ESG analyst instructions
│   │   ├── ollama-client.js              # Ollama + OpenAI + OpenRouter
│   │   ├── db.js / pg-client.js          # SQLite & Postgres adapters
│   │   ├── sql-sanitize.js               # ranking & column safety
│   │   ├── report-sources.js             # PDF URLs & citation fields
│   │   ├── page-index.js                 # PDF page → metric mapping
│   │   ├── pdf-verifier.js               # post-answer citation checks
│   │   ├── share-breakdown.js            # % with numerator/denominator
│   │   ├── auth.js / user-chats.js       # Firebase + session chats
│   │   ├── hf-pdfs.js / r2-pdfs.js       # PDF hosting helpers
│   │   └── public/                       # Gemini-style chat UI
│   └── scripts/                          # ingest, eval, upload, tests
├── data/
│   ├── database.db                       # local SQLite
│   ├── xbrl/ · pdf/                     # raw filings
│   ├── nse_sustainability_metadata.json
│   └── evaluation_* / model_*            # quality reports
```

3.1 Core runtime modules

MODULE	RESPONSIBILITY
<code>server.js</code>	HTTP API: status, config, auth, sessions, XBRL upload, SSE chat, static UI, local PDF serve.
<code>agent.js</code>	Defines tools, runs the think → tool → think loop, truncates SQL results, upgrades citations, normalizes chart JSON.
<code>db.js</code>	Schema init, insert/query reports, company resolution, metric page indexing; switches to Postgres when <code>DATABASE_URL</code> is set.
<code>ollama-client.js</code>	Unified chat completion client with retries, fallbacks, and provider priority: OpenAI → OpenRouter → Ollama.
<code>sql-sanitize.js</code>	Rewrites ORDER BY for share metrics, filters invalid ranking rows, blocks unsupported metric substitutes.
<code>report-sources.js</code> + <code>page-index.js</code>	Attach PDF URLs and page-level <code>_citation</code> fields to tool outputs.
<code>auth.js</code> + <code>user-chats.js</code>	Firebase ID token verification, httpOnly session cookies, per-user chat persistence.

3.2 Agent tools (LLM-callable)

TOOL	WHEN USED	RETURNS
<code>execute_sql_query</code>	Rankings, comparisons, aggregates, filters	SELECT rows (capped), enriched with citations when possible
<code>get_company_report</code>	Deep single-company qualitative + quantitative detail	Full report JSON + share breakdowns + sources
<code>list_companies</code>	Discovery (small corpora only)	Company name list or guidance to use LIKE
<code>list_company_reports</code>	Which years exist for a company	Year / filename / custom flag

4. How a Query Works

End-to-end path for a typical question such as “*Top 5 companies by renewable energy share in 2025*”:

1

User sends the question

The UI posts to `POST /api/chat` with the message and recent chat history. The response is an **SSE** stream so status updates appear live.

2

Agent builds the LLM context

`runAgent` loads the compact system prompt (schema + rules), trims history to the last few turns, and registers the four tools.

3

Model chooses a tool

For rankings it typically emits one `execute_sql_query` call, e.g. selecting `company`, `year`, `renewable_energy_share` with `WHERE year = 2025`, non-null filters, and `ORDER BY ... DESC LIMIT 5`.

4

SQL is sanitized and executed

Only `SELECT` is allowed. Company equality is relaxed to `LIKE '%...%'`. Unknown columns are rejected. Ranking queries are rewritten so `NULL/0/invalid` share rows cannot “win.” Row count is truncated for context limits.

5

Results are enriched

Rows get PDF URLs, share breakdowns (e.g. GJ renewable of GJ total), and page citations when `metric_pages_json` has a confident match. Progress events update the UI (“Comparing...”, “Gathered data...”).

6

Final answer is generated

The LLM receives tool output and writes a structured analysis (Executive Summary, Key Findings, Analysis, Recommendation) plus an optional `json-chart` block. Citations appear inline only when verified fields exist.

7

Stream completes & chat is stored

SSE sends `status: done`. Guests may use `localStorage`; signed-in users persist sessions via Firebase-backed tables. A copy may also land in `data/chats.json` for ops debugging.

4.1 Safety rules that shape answers

- **Schema-bound:** If the user asks for a metric that is not a column (e.g. “disabled female workers”), the agent must say it is unavailable — not substitute a related field.
- **Year exactness:** Asking for 2025 must not silently return 2026.

- **Citation honesty:** Aggregates and sector averages never get fake PDF page citations.
- **Share display:** Percentages include numerator/denominator from `share_breakdown`, without showing raw formulas.

4.2 Example SQL the agent might emit

```
SELECT company, year, renewable_energy_share,
       renewable_energy_consumption, energy_consumption
FROM reports
WHERE year = 2025
      AND renewable_energy_share IS NOT NULL
      AND renewable_energy_share > 0
ORDER BY renewable_energy_share DESC
LIMIT 5;
```

5. Project Setup

5.1 Prerequisites

- Node.js 18+ (project developed on Node 21)
- npm
- Optional for local LLM: Ollama with `qwen2.5:7b-instruct`
- Optional for cloud: OpenAI or OpenRouter API key
- Optional for production DB: Neon Postgres connection string
- Optional for auth: Firebase project (web config + service account)

5.2 Install dependencies

```
cd stage3
npm install
# If downloading NSE reports with Puppeteer:
npm run install-browser
```

5.3 Configure environment

Copy `.env.example` to `.env` and fill the values you need:

VARIABLE	PURPOSE
PORT	Web server port (default 3000)
DB_PATH	Local SQLite path (data/database.db)
DATABASE_URL	Neon Postgres URL — when set, replaces SQLite
OLLAMA_HOST / OLLAMA_MODEL	Local inference endpoint and model
OPENAI_API_KEY / OPENROUTER_API_KEY	Cloud LLM (preferred when present)
FIREBASE_* / SESSION_SECRET	Authentication and signed cookies
HF_TOKEN / HF_DATASET_REPO	Public PDF hosting on Hugging Face
R2_*	Optional Cloudflare R2 PDF hosting
AGENT_MAX_ITERATIONS	Tool-loop budget (default 3)

5.4 Build the database from XBRL

```
# Optional: download filings from NSE
npm run download-reports

# Parse XBRL → reports table
npm run preprocess

# Optional: enrich PDF/XBRL source URLs + page index
npm run enrich:sources

# Optional: migrate SQLite → Neon Postgres
npm run migrate:postgres
```

5.5 Run the app

```
# Start API + UI
npm start
# → http://localhost:3000

# Quality / regression scripts
npm run evaluate
npm run test:citations
npm run test:all:quick
```

First-time tip. If data/database.db already exists in the repo, you can skip preprocess and

start the server immediately. Cloud LLM keys avoid needing a local Ollama install.

6. Hosting & Deployment

The app is designed for **Vercel serverless** hosting of `agent/src/server.js`, with configuration in `vercel.json`. Critical data files (SQLite DB when used, metadata JSON, URL maps, and the public UI) are included in the build.

6.1 Recommended production topology

Compute & API

- Vercel Node serverless function
- Routes: all traffic → `server.js`
- On Vercel, SQLite is copied to `/tmp` if used

Database

- **Neon Postgres** via `DATABASE_URL`
- Survives serverless ephemeral disks
- Same `reports` schema as SQLite

Auth & sessions

- Firebase Authentication (client)
- Admin SDK verifies ID tokens
- `httpOnly` cookie sessions

PDF sources

- Hugging Face Datasets (preferred free public links)
- Cloudflare R2 as alternate
- URL maps committed as JSON

6.2 Deployment steps (checklist)

1. Create a Neon project and set `DATABASE_URL`.
2. Run `npm run migrate:postgres` (or preprocess against Postgres) so `reports` is populated.
3. Upload PDFs: `npm run upload:hf` (or `upload:r2`) and commit the URL map.
4. Configure Firebase web + service account env vars and `SESSION_SECRET`.
5. Set at least one LLM key (`OPENAI_API_KEY` or `OPENROUTER_API_KEY`) — local Ollama is not available on Vercel.
6. Connect the Git repo to Vercel; env vars mirror `.env.example`.
7. Deploy; verify `/api/status`, `/api/config`, sign-in, and a sample chat.

6.3 Local vs hosted comparison

CONCERN	LOCAL DEVELOPMENT	HOSTED (VERCEL)
Database	SQLite file	Neon Postgres
LLM	Ollama (free, private)	OpenAI / OpenRouter
PDFs	/local-pdf/...	Hugging Face or R2 public URLs
Auth	Optional	Recommended (Firebase)
Process model	Long-running Node	Serverless cold starts

7. Tools Used and Why

TOOL / LIBRARY	ROLE	WHY IT WAS CHOSEN
Node.js + Express 5	API & static UI	Simple fullstack JS; SSE streaming; easy Vercel Node deploy.
Ollama	Local LLM	Zero API cost for development; tool-calling with open models like Qwen 2.5.
OpenAI / OpenRouter	Cloud LLM	Higher quality & reliability in production; OpenRouter adds model fallbacks.
SQLite (<code>sqlite</code> / <code>sqlite3</code>)	Local DB	Zero-ops, file-backed, perfect for offline preprocess and demos.
Neon Postgres (<code>pg</code>)	Production DB	Serverless-friendly Postgres; shared schema with SQLite adapter.
fast-xml-parser	XBRL ingest	Fast namespace-aware parsing of BRSR XML into JSON.
Puppeteer	NSE download	Automates browser session for NSE sustainability report downloads.
pdfjs-dist	PDF text index	Page-level text extraction for citation page numbers.
Firebase Admin	Auth	Managed identity; ID tokens verified server-side; familiar web SDK.
Hugging Face Hub	PDF hosting	Free public dataset hosting for citable PDF URLs.
Cloudflare R2 (S3 API)	Alt. PDF hosting	Object storage with public base URL when HF is not enough.
Vercel	Hosting	Git-based deploys; Node serverless matches Express export.
dotenv / multer / jwt / cookie-parser	Config, uploads, sessions	Standard, minimal glue for env, XBRL upload, and cookie sessions.

7.1 Why a tool-calling agent (not only RAG)

BRSR questions are often *analytical* (rank, average, compare) rather than paragraph lookup. SQL over a wide `reports` table answers those cleanly. Tool calling lets the model compose queries while application code enforces safety. PDF page indexing then adds transparency for single-company metric claims.

8. What I Learned from This Project

1. **Grounding beats eloquence.** An LLM that speaks confidently about Scope 1 emissions is useless without a SELECT path and “never invent numbers” rules. Schema-bound prompts plus SQL column allowlists cut hallucinations more than longer system prompts alone.
2. **Tool calling needs application guardrails.** Models emit exact `company = 'HDFC Bank'`, invent columns, or rank NULL as zero. Fuzzy LIKE rewrites, unknown-column errors, and ranking sanitizers turned “almost right” SQL into trustworthy answers.
3. **Citations are a product feature, not a markdown trick.** Linking to a PDF is easy; linking to the *correct page* requires indexing, scoring, and refusing to cite aggregates. Honesty about “no citation available” builds more trust than fake page numbers.
4. **Local vs cloud is a continuum.** Ollama is ideal for iteration cost and privacy; production needs cloud models and a durable database. Designing one adapter layer (`ollama-client.js` , `db.js`) made that swap a configuration change.
5. **Streaming UX matters for agent latency.** Multi-step tool loops feel slow if the UI waits for a single JSON blob. SSE progress (“Querying BRSR...”, “Gathered data...”) keeps users oriented during 5–30s runs.
6. **Evaluation is part of the system.** Scripts for smoke tests, citation tests, model comparisons, and LLM-as-judge scoring made regressions visible when prompts or sanitizers changed.
7. **ESG domain knowledge belongs in the prompt and schema.** Year ↔ fiscal-year mapping, intensity metrics, and share breakdowns are domain rules. Encoding them centrally (system prompt + `share-breakdown.js`) keeps answers consistent across models.
8. **Deployment constraints reshape architecture.** Serverless filesystems are ephemeral; SQLite demos do not survive. Neon, external PDF hosting, and Firebase sessions were necessary to turn a laptop agent into a hosted product.

9. Evaluation, Testing & Quality

Quality work lives under `agent/scripts/` and `data/` reports:

COMMAND / ARTIFACT	PURPOSE
<code>npm run evaluate</code>	Run the agent on a fixed question bank
<code>npm run evaluate:judge</code>	Score answers with a judge model
<code>npm run compare:models</code>	Side-by-side model comparison
<code>npm run test:citations</code>	Verify PDF page citation correctness
<code>npm run test:all</code>	Full regression suite
<code>complex_queries_test_bank.md</code>	Hard analytical questions for manual/CI use
<code>data/evaluation_report.md</code> etc.	Generated quality reports and model outputs

10. Appendix — Schema, Scripts & API

10.1 Primary `reports` columns (queryable)

Identity: `company` , `year` , `sector` , `industry` . Emissions: `scope1_emissions` , `scope2_emissions` , `scope3_emissions` , intensities. Energy / water / waste: consumption, renewable share, intensities. Diversity: female employee/board counts and shares. Other: `safety_ltifr` , `total_revenue` , qualitative text fields, `data_json` , `pdf_url` , `xbrl_url` , `metric_pages_json` .

10.2 Key npm scripts

SCRIPT	DESCRIPTION
<code>npm start</code> / <code>npm run dev</code>	Run the web server
<code>npm run preprocess</code>	XBRL → database
<code>npm run download-reports</code>	Fetch NSE BRSR packages
<code>npm run upload:hf</code> / <code>upload:r2</code>	Publish PDFs
<code>npm run migrate:postgres</code>	SQLite → Neon
<code>npm run evaluate*</code> / <code>test:*</code>	Quality & regression

10.3 Main HTTP endpoints

METHOD & PATH	DESCRIPTION
GET /api/status	Report count and inventory
GET /api/config	Provider, model, Firebase public config
POST /api/chat	SSE agent conversation
POST /api/auth/firebase	Exchange ID token → session cookie
GET/POST /api/sessions*	Authenticated chat persistence
POST /api/upload-custom	Upload a custom XBRL file
POST /api/delete-report	Remove a report row
/local-pdf/*	Serve local PDF files for citations

10.4 Reports by year (corpus snapshot)

YEAR	REPORT COUNT	NOTES
2020	2	Sparse early coverage
2021	183	
2022	1,133	
2023	1,385	
2024	1,420	Largest year
2025	1,320	FY 2024–25 mapping in prompt
2026	352	Partial / ongoing